	`remove(element)`
	`update(element, newPriority)`
	`contains(element)`
	`min()`
	`max()`
	`}`

Contract with client	Entries are kept sorted by element (*aka* key), but at any time the `top()` and `peek()` methods can return the element with the highest priority

To this goal, we can imagine an extension of priority queues that also keep its elements sorted: in the rest of the chapter, we will use the term “key” to refer to elements, and associate a priority to each element.

With respect to the `PriorityQueue` ADT that we have introduced in chapter 2, this new `SortedPriorityQueue` class has a few new methods: a search method to find a given key in the container, and two methods returning the minimum and maximum key stored.

If you look at appendix C, you can see that these three operations are usually implemented by many basic data structures, like linked lists, arrays or trees (some of them in linear time, some others with better performance, offering logarithmic methods).

Thus, we can think about our `SortedPriorityQueue` as a melting pot between two different containers, integrating both data structures’ characteristics and providing methods from both of them; for instance, it could be though as a fusion between heaps and linked lists, or…

3.3 Treap

… Or between a tree and a heap!